

SIR Model on a Network

EdgeBasedModels.jl Vignette 01

Simon Frost

2026-05-14

Table of contents

Introduction	1
Setup	2
Defining the network	2
Building the compact SIR model	2
Solving the ODE	3
Expanded formulation	6
Effect of transmission rate	9
Simulation validation	11
Summary	13

Introduction

Edge-based compartmental models (EBCMs) provide an exact, deterministic framework for tracking epidemics on configuration-model networks. Instead of simulating individual transmission events, we track the probability $\theta(t)$ that a random edge in the network has *not* transmitted infection by time t .

Combined with the network's probability generating function (PGF) $\psi(x) = \sum_k p_k x^k$ — which encodes the degree distribution — we can derive the fraction susceptible as $S(t) = \psi(\theta(t))$ and the fraction recovered $R(t)$ from a small system of ODEs.

The compact formulation (Miller 2011) for SIR on a configuration model network requires just two ODEs:

$$\dot{\theta} = -\beta\theta + \beta \frac{\psi'(\theta)}{\psi'(1)} + \gamma(1 - \theta)$$

$$\dot{R} = \gamma(1 - \psi(\theta) - R)$$

where β is the per-edge transmission rate and γ is the recovery rate.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using OrdinaryDiffEq
using Symbolics
using Plots
```

Defining the network

We model a Poisson (Erdős–Rényi) network with mean degree $\kappa = 5$. The PGF is $\psi(x) = e^{\kappa(x-1)}$.

```
@parameters β γ κ
pgf = poisson_pgf(κ)
```

```
DegreePGF(z, exp((-1 + z)*κ))
```

We can verify the mean degree:

```
md = mean_degree(pgf)
println("Mean degree: ", md)
```

```
Mean degree: exp(0)*κ
```

Building the compact SIR model

The `build_sir` function constructs the edge-based ODE system. The `:compact` form produces just 2 ODEs.

```
model_compact = build_sir(pgf, β, γ; form = :compact)
eqs = ModelingToolkit.equations(model_compact.system)
println("Number of ODEs: ", length(eqs))
for eq in eqs
    println(" ", eq)
end
```

Number of ODEs: 2

$$\text{Differential}(t, 1)(R(t)) \sim (1 - R(t) - \exp((-1 + \theta(t))\kappa) * (1 - \rho)) * \gamma$$
$$\text{Differential}(t, 1)(\theta(t)) \sim (\theta(t) * \exp(0) * \beta - (1 - \theta(t)) * \exp(0) * \gamma - \exp((-1 + \theta(t))\kappa) * \beta * (1 - \rho))$$

The model returns an `EdgeModelSystem` with the compiled system, a dictionary of state variables, and a dictionary of observables:

```
println("State variables: ", keys(model_compact.variables))
println("Observables:      ", keys(model_compact.observables))
```

State variables: [:R, :θ]

Observables: [:I, :ψ_θ, :S]

Solving the ODE

We anchor this network example to the well-mixed reference $R_0 = 2$ with $\gamma = 0.25$ and 1% initial infection. For a Poisson configuration model, $\kappa_{\text{excess}} = \kappa$, so $T = R_0/\kappa$ and $\beta = T\gamma/(1 - T)$.

Note

$R_0 = 2$ anchor. Here $\kappa = 5$, so $T = 2/5$ and the comparable per-edge transmission rate is $\beta = 1/6 \approx 0.1667$, not the well-mixed mass-action value 0.5.

```
γ_val = 0.25
R0_target = 2.0
seed_fraction = 0.001
κ_val = 5.0
κ_excess = κ_val
T_val = R0_target / κ_excess
β_val = T_val * γ_val / (1 - T_val)
tspan = (0.0, 40.0)
prob = ODEProblem(
    model_compact.system,
    merge(
        default_initial_conditions(model_compact; seed_fraction = seed_fraction),
        Dict{β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val},
    ),
    tspan,
)
sol = solve(prob, Tsit5(); saveat = 0.5)
```

```

retcode: Success
Interpolation: 1st order linear
t: 81-element Vector{Float64}:
 0.0
 0.5
 1.0
 1.5
 2.0
 2.5
 3.0
 3.5
 4.0
 4.5
 ⋮
36.0
36.5
37.0
37.5
38.0
38.5
39.0
39.5
40.0
u: 81-element Vector{Vector{Float64}}:
 [0.0, 1.0]
 [0.0001443066420063512, 0.9999073735034281]
 [0.0003321822414562357, 0.9997933627469192]
 [0.0005724277887093543, 0.9996530647361639]
 [0.0008759757150911707, 0.99948046911652]
 [0.001256342187138002, 0.9992682162790546]
 [0.0017301617006247436, 0.9990073125847139]
 [0.0023178470957322585, 0.9986867811738597]
 [0.0030443808440527003, 0.9982932453378268]
 [0.003940113081970378, 0.9978105103839388]
 ⋮
 [0.7827238838058135, 0.6832800031238893]
 [0.7841805223121907, 0.683035527286805]
 [0.7854919643696453, 0.6828158250672444]
 [0.7866739193257721, 0.6826208291605494]
 [0.7877389161730758, 0.6824484624230917]
 [0.7886980554934381, 0.6822961885241878]
 [0.7895615517365182, 0.6821616804278492]
 [0.7903387332197528, 0.6820428203927811]

```

```
[0.7910380421283557, 0.6819376999723834]
```

Now we extract the population-level quantities using the model's named variables and observables rather than assuming a positional ordering in the ODE state vector. For a Poisson PGF, $S(t) = \psi(\theta(t)) = e^{\kappa(\theta-1)}$:

```
S_vals = compartment(sol, model_compact, :S)
I_vals = compartment(sol, model_compact, :I)
R_vals = compartment(sol, model_compact, :R)
```

81-element Vector{Float64}:

```
0.0
0.0001443066420063512
0.0003321822414562357
0.0005724277887093543
0.0008759757150911707
0.001256342187138002
0.0017301617006247436
0.0023178470957322585
0.0030443808440527003
0.003940113081970378
⋮
0.7827238838058135
0.7841805223121907
0.7854919643696453
0.7866739193257721
0.7877389161730758
0.7886980554934381
0.7895615517365182
0.7903387332197528
0.7910380421283557
```

```
plot(sol.t, S_vals, label = "S", linewidth = 2, color = :blue)
plot!(sol.t, I_vals, label = "I", linewidth = 2, color = :red)
plot!(sol.t, R_vals, label = "R", linewidth = 2, color = :green)
xlabel!("Time")
ylabel!("Fraction of population")
title!("SIR on Poisson network ( $\kappa=5$ ,  $R_0=2$ ,  $\gamma=0.25$ )")
```

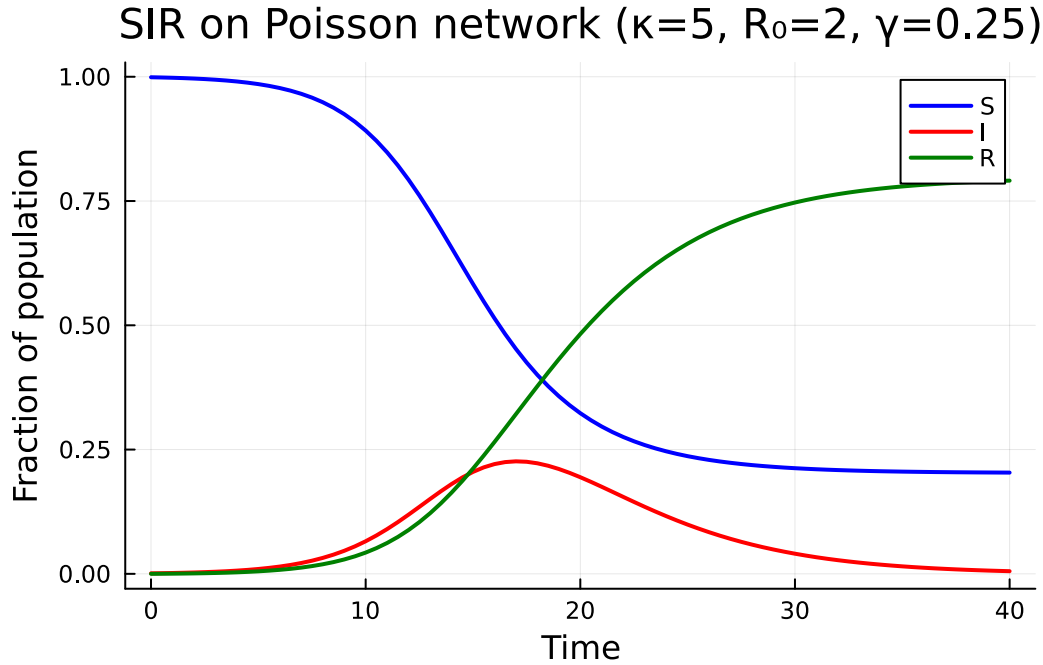


Figure 1: SIR epidemic on a Poisson network (compact formulation).

Expanded formulation

The expanded form tracks edge-level probabilities φ_I and φ_R explicitly, plus the population-level R . This form generalises to arbitrary disease progression graphs.

```
model_expanded = build_sir(pgf,  $\beta$ ,  $\gamma$ ; form = :expanded)
eqs_exp = ModelingToolkit.equations(model_expanded.system)
println("Number of ODEs: ", length(eqs_exp))
for eq in eqs_exp
    println(" ", eq)
end
```

Number of ODEs: 5

```
Differential(t, 1)(pop_R(t)) ~ pop_I(t)* $\gamma$ 
Differential(t, 1)(pop_I(t)) ~ -pop_I(t)* $\gamma$  + exp((-1 +  $\theta(t)$ )* $\kappa$ )*phi_I(t)* $\beta$ * $\kappa$ *(1 -  $\rho$ )
Differential(t, 1)(phi_R(t)) ~ phi_I(t)* $\gamma$ 
Differential(t, 1)(phi_I(t)) ~ (exp(0)*phi_I(t)*( $\beta$  +  $\gamma$ ) - exp((-1 +  $\theta(t)$ )* $\kappa$ )*phi_I(t)* $\beta$ * $\kappa$ *(1 -  $\rho$ ))
Differential(t, 1)( $\theta(t)$ ) ~ -phi_I(t)* $\beta$ 
```

```
println("State variables: ", keys(model_expanded.variables))
println("Observables:      ", keys(model_expanded.observables))
```

```
State variables: [:R, :φ_I, :pop_I, :pop_R, :φ_R, :θ]
Observables:     [:I, :φ_S, :S, :edge_hazard, :excess_hazard]
```

Let us solve the expanded model and verify it gives the same epidemic curve:

```
ic_exp = default_initial_conditions(model_expanded; seed_fraction = seed_fraction)
prob_exp = ODEProblem(
    model_expanded.system,
    merge(ic_exp, Dict{β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val}),
    tspan,
)
sol_exp = solve(prob_exp, Tsit5(); saveat = 0.5)
```

```
retcode: Success
Interpolation: 1st order linear
t: 81-element Vector{Float64}:
 0.0
 0.5
 1.0
 1.5
 2.0
 2.5
 3.0
 3.5
 4.0
 4.5
 ⋮
36.0
36.5
37.0
37.5
38.0
38.5
39.0
39.5
40.0
u: 81-element Vector{Vector{Float64}}:
 [0.0, 0.001, 0.0, 0.0010000000000000009, 1.0]
```

```
[0.00014430664466060414, 0.0013182555929231552, 0.00013893974666429076, 0.0012309959931432
[0.0003321821817661868, 0.0016994376643806508, 0.0003099558273684049, 0.001515026800532830
[0.0005724278469447895, 0.002159011783833065, 0.0005204029420845671, 0.0018641013939702438
[0.0008759755245426545, 0.002715713064586523, 0.0007792961492561455, 0.00229286167370227, 0
[0.0012563423418836285, 0.003392238792016434, 0.0010976757036694439, 0.002819121627784325
[0.0017301607135635916, 0.004216023901332757, 0.0014890301610309047, 0.0034644676798448425
[0.0023178504578487834, 0.005220199627847355, 0.001969830835161054, 0.00425499869376105, 0
[0.003044381011402437, 0.006444576320585768, 0.0025601315092720943, 0.0052220714832013826,
[0.0039401158105192065, 0.0079367503052407, 0.0032842355238153704, 0.006403140242734291, 0
?]
[0.782710296293347, 0.012261542974824182, 0.4750859900451043, 0.003161855859663926, 0.68327
[0.7841668534879456, 0.011059199550726241, 0.47545819212570567, 0.0027957328291622163, 0.68
[0.7854803945031333, 0.009970291577553364, 0.4757874754221219, 0.002471560377150037, 0.6828
[0.7866641867434654, 0.008984781631728443, 0.47607844328885995, 0.0021848962270936897, 0.68
[0.7877305246909642, 0.008093368871331552, 0.47633537647955987, 0.0019315994296958687, 0.68
[0.7886907299051198, 0.007287489036099925, 0.4765622331469943, 0.001707830362895659, 0.6822
[0.7895551510228892, 0.006559314447428004, 0.4767626488430691, 0.0015100507318685427, 0.682
[0.7903331637586972, 0.005901754008367396, 0.4769399365188228, 0.0013350235690263942, 0.682
[0.7910331565859934, 0.00530835122118558, 0.4770969100547465, 0.0011799910492679764, 0.6819
```

```
S_exp = compartment(sol_exp, model_expanded, :S)
I_exp = compartment(sol_exp, model_expanded, :I)
```

81-element Vector{Float64}:

```
0.001
0.0013182555929231552
0.0016994376643806508
0.002159011783833065
0.002715713064586523
0.003392238792016434
0.004216023901332757
0.005220199627847355
0.006444576320585768
0.0079367503052407
?]
0.012261542974824182
0.011059199550726241
0.009970291577553364
0.008984781631728443
0.008093368871331552
0.007287489036099925
0.006559314447428004
```



```
0.005901754008367396
0.00530835122118558
```

```
plot(sol_exp.t, I_exp, label = "I (expanded)", linewidth = 2, color = :red)
scatter!(sol.t[1:4:end], I_vals[1:4:end], label = "I (compact)",
        markershape = :circle, markersize = 4, markerstrokewidth = 0,
        color = :black)
xlabel!("Time")
ylabel!("Fraction infected")
title!("Compact vs Expanded Formulation")
```

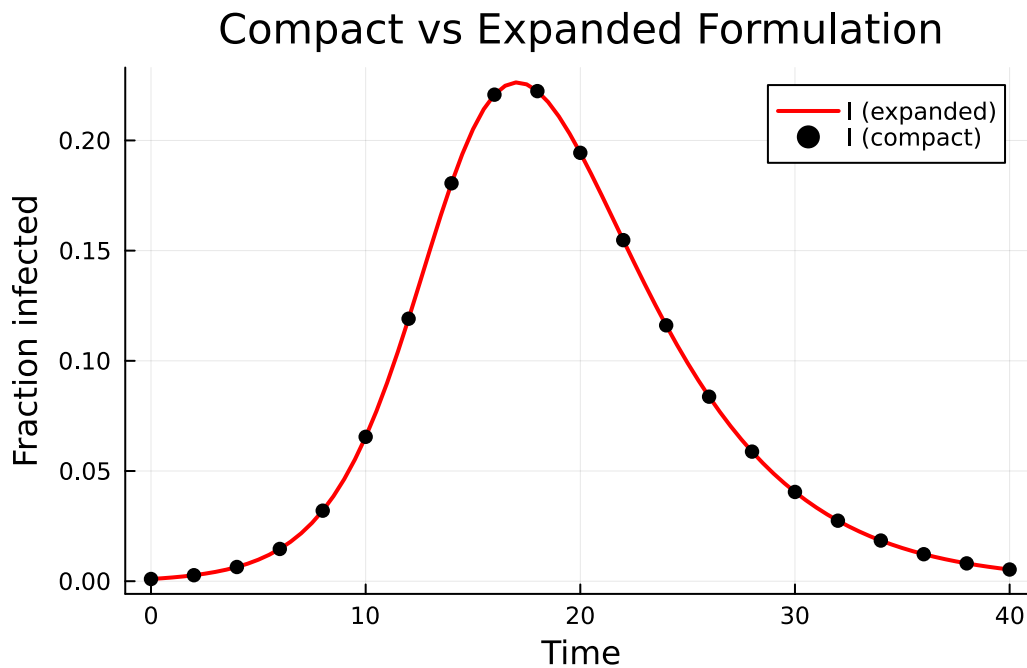


Figure 2: Expanded (line) and compact (markers) formulations overlap exactly.

The two formulations give identical epidemic curves, confirming that the compact form (2 ODEs) is an exact reduction of the expanded form (5 ODEs after `mtkcompile`) for SIR.

Effect of transmission rate

```
betas =  $\beta_{\text{val}}$  .* [0.5, 1.0, 1.5, 2.0]
p = plot(xlabel = "Time", ylabel = "Fraction infected",
```

```

        title = "Effect of  $\beta$  on epidemic dynamics ( $\kappa=5$ , central  $R_0=2$ )")

for b in betas
    prob_b = ODEProblem(
        model_compact.system,
        merge(
            default_initial_conditions(model_compact; seed_fraction = seed_fraction),
            Dict( $\beta \Rightarrow b$ ,  $\gamma \Rightarrow \gamma\_val$ ,  $\kappa \Rightarrow \kappa\_val$ ),
        ),
        tspan,
    )
    sol_b = solve(prob_b, Tsit5(); saveat = 0.5)
    I_b = compartment(sol_b, model_compact, :I)
    plot!(p, sol_b.t, I_b, label = " $\beta = \$b$ ", linewidth = 2)
end
p

```

Effect of β on epidemic dynamics ($\kappa=5$, central R_0 :

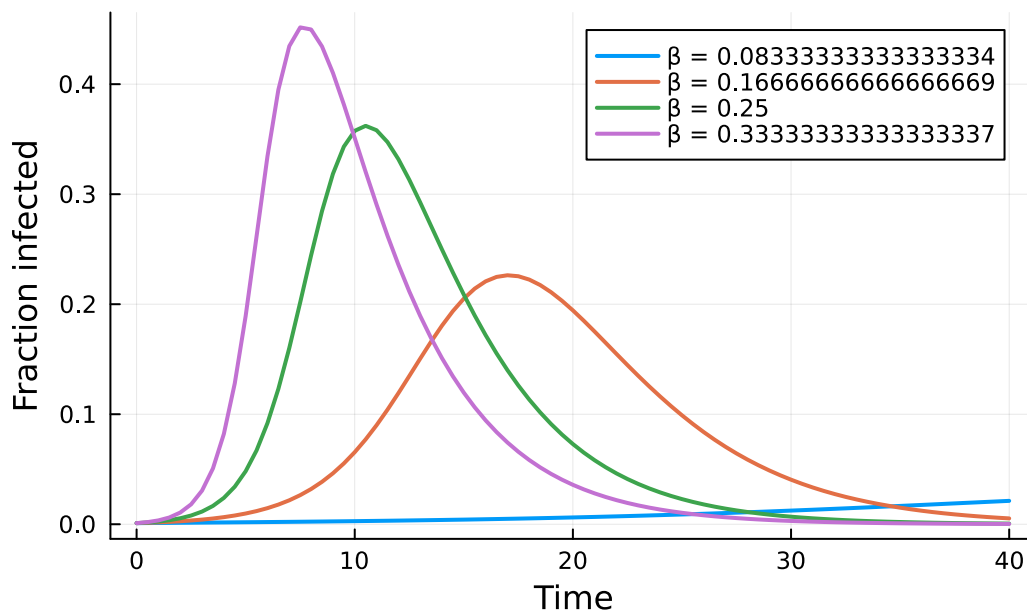


Figure 3: Effect of transmission rate β on the epidemic curve.

Simulation validation

To check that the deterministic EBCM is consistent with stochastic dynamics on a finite network, we simulate the same SIR process with the Gillespie SSA provided by [NetworkOutbreaks.jl](#) on Erdős–Rényi graphs with the same mean degree $\kappa = 5$, and overlay the ensemble mean and $\pm 1\sigma$ band on the EBCM prediction.

To reduce conditioning on a single graph realisation, we average across several independent host graphs as well as across stochastic epidemic realisations on each.

```
using NetworkOutbreaks
using Graphs
using StableRNGs
using Statistics

N = 1000
n_graphs = 5
nsims_per_graph = 20
rng_host = StableRNG(20240501)

prog = sir_model()
no_model = OutbreakModel(prog, Dict{:β ⇒ β_val, :γ ⇒ γ_val}))

tgrid = collect(0.0:0.5:40.0)
I_idx = no_model.index_of[:I]

I_runs = Matrix{Float64}(undef, n_graphs * nsims_per_graph, length(tgrid))
row = 1
for gi in 1:n_graphs
    g = erdos_renyi(N, κ_val / (N - 1); rng = rng_host)
    spec = OutbreakSpec(model = no_model, network = g,
                        initial = SeedFraction(:I ⇒ seed_fraction),
                        tspan = (0.0, 40.0))
    ens = simulate_ensemble(spec; nsims = nsims_per_graph,
                            seed = 20240501 + 1000 * gi,
                            parallel = true)
    for traj in ens.trajectories
        for (j, t) in enumerate(tgrid)
            I_runs[row, j] = Float64(state_at(traj, t)[I_idx])
        end
        row += 1
    end
end
end
```

```

μI = vec(mean(I_runs; dims = 1)) ./ N
σI = vec(std(I_runs; dims = 1)) ./ N

```

81-element Vector{Float64}:

```

0.0
0.0009987365756167878
0.0017132578688722035
0.0026180993954478594
0.0036080017693561027
0.005011340674264259
0.005970389899793443
0.007261876374935647
0.009504337127986585
0.010943269690323432
⋮
0.009475001665733711
0.00860877518007121
0.007967737977923751
0.007418077836466722
0.006979045114273922
0.006279186397789273
0.005710171679210758
0.00526481555501072
0.004582311180788114

```

```

plot(tgrid, μI, ribbon = σI, label = "Gillespie SSA (mean ± 1σ)",
     color = :red, fillalpha = 0.2, linealpha = 0.6, linewidth = 1)
plot!(sol.t, I_vals, label = "EBCM compact",
     color = :red, linewidth = 2)
xlabel!("Time")
ylabel!("Fraction infected")
title!("EBCM vs Gillespie SSA (Poisson κ=5, R₀=2, N=$N)")

```

EBCM vs Gillespie SSA (Poisson $\kappa=5$, $R_0=2$, $N=10$)

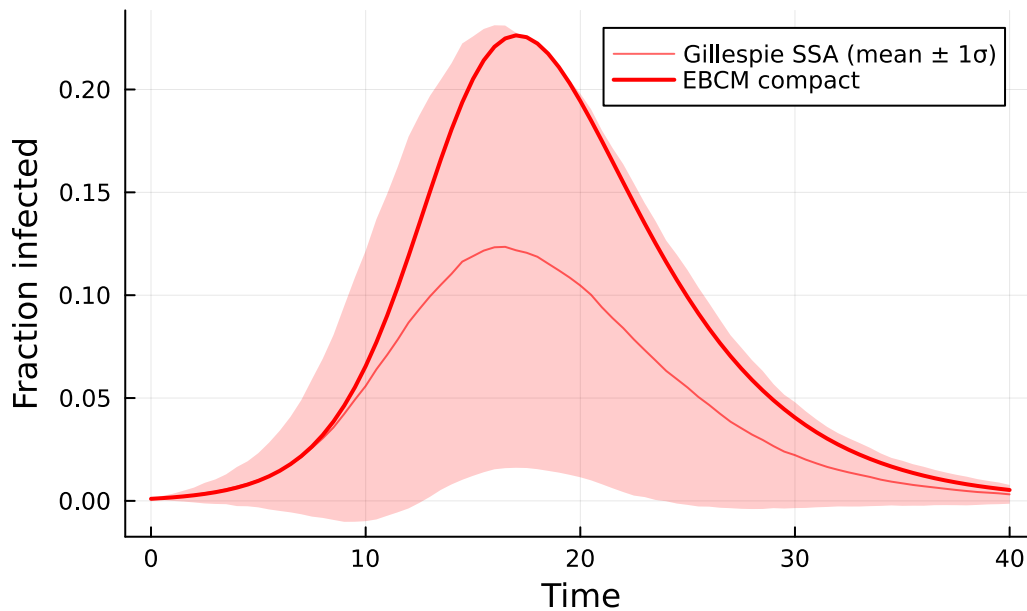


Figure 4: EBCM prediction (red line) versus Gillespie SSA on Erdős–Rényi graphs (red ribbon: mean $\pm 1\sigma$ across $n_{\text{graphs}} \times n_{\text{sims_per_graph}}$ runs at $N=1000$).

The deterministic EBCM curve sits inside the stochastic ribbon throughout the epidemic, confirming that the configuration-model closure correctly predicts the mean prevalence on this network ensemble.

Summary

- The compact formulation produces 2 ODEs and is the most efficient for SIR on static networks.
- The expanded formulation produces $1 + 2M$ ODEs (where M is the number of non-susceptible stages, so 5 ODEs for SIR) but generalises to arbitrary disease progression graphs (SEIR, multi-stage, etc.).
- Both formulations give identical results for SIR.
- The PGF encodes the network’s degree distribution — we used a Poisson PGF here, but the package supports arbitrary degree distributions via `polynomial_pgf`.
- The EBCM ODE prediction agrees with Gillespie SSA on finite Erdős–Rényi graphs with matching mean degree.